Progressive Retrieval Attention with Ollama: Transparent Context Optimization for Local-Model Applications

Jorge Simão

September 2026

Abstract

Ollama combines local-model packaging, lifecycle management, and a stable application API. Progressive Retrieval Attention (PRA) should preserve that product surface: ordinary installations receive selected-text E0, while a PRA-aware model backend may negotiate native E2 without requiring a second attention implementation in Ollama. We implement this capability-negotiated adapter, pin a current source audit, and test Windows and Apple Metal daemons. Across Qwen3-8B on M5 and Qwen3-14B on M4 Pro, selected prompts use 10.0–42.4% of full-context tokens. At 14B, median time to first token falls from 1731 to 1044 ms on HotpotQA and from 6257 to 847 ms on QASPER; 2Wiki again reverses direction under warm reuse. Its small-cohort F1 is lower on HotpotQA and QASPER and equal on 2Wiki, separating transport benefit from quality preservation. Keeping Qwen3-8B alive reduced request latency from 5.84 s cold to 0.714 s across five warm requests (8.18×). Stock Ollama exposed no native endpoint, so AUTO correctly remained E0. We then connect the product boundary to a pinned llama.cpp sequence-attachment mechanism: 10/10 CPU/Metal runs have exact schedule-matched logits and persistent decode. A versioned, model-bound Ollama handshake accepts that complete receipt, rejects incomplete or stale receipts, and invalidates state on model switch and unload. A live M4 Pro run then delegates 25/25 requests against Ollama's exact Qwen2.5-0.5B model blob. E0 and E2 both answer 25/25 correctly; warm E2 reproduces 25/25 token sequences. Mean latency is 147.8 ms for E0, 78.3 ms for cold-resource E2, and 54.1 ms for warm-resource E2. This closes product delegation to a pinned sidecar, not native execution inside the stock Ollama runner.

1 Introduction

Ollama's value is operational simplicity: a model name, one daemon, and a familiar API. A PRA integration that requires every application to understand engine-specific K/V state would undermine that value. We instead treat Ollama as the product and model-lifecycle layer. PRA owns typed records, authorization, selection, sessions, and storage policy. The active model runner owns attention and physical cache state.

This division creates one hard rule: deeper support is negotiated, never inferred from a version string or from Ollama's historical relation to a particular backend. Current Ollama may select different runners for different models and platforms. An application-facing adapter must therefore be able to downgrade safely.

Our contributions are an Ollama-native HTTP adapter, model- and artifact-bound invalidation, a mechanism-specific backend handshake, a live negotiated llama.cpp executor, selected-text and lifecycle measurements, and tests that keep the advertised integration level honest.

2 PRA Levels and Ownership

E0 serializes selected evidence into visible messages. E1 transports logical resource identities without changing attention. E2 consumes detached selected native K/V. E3 additionally requires scheduler-owned placement, sharing, prefetch, and eviction. The adapter reports the deepest implemented level, not the deepest level described by the protocol.

3 Pinned Architecture Audit

The source audit pins Ollama commit `e37a00a8fa94fca07cefd544bffc9d8997ebcd44`. The measured daemon is version 0.6.8; these are recorded separately because behavior observed in one must not be projected onto the

Existing Ollama client → PRA gateway/SDK → capability probe.
No native handshake → selected records become an ordinary system message → `/api/chat` (E0).
Validated backend handshake → resource/version/model fingerprint → backend-native PRA executor (E2/E3).
Unload or model switch → invalidate incompatible native state; Ollama continues to own runner load, keep-alive, and scheduling.

Figure 1: Gateway-first integration. Native support is inherited from the active backend rather than reimplemented in Ollama.

other. The audited code exposes chat/generate, model inspection, installed/running model lists, keep-alive, and runner scheduling. Backend choice is an implementation decision, so the PRA capability cannot be deduced from the model package alone.

The public API exposes useful timing counters: load, prompt evaluation, token evaluation, and total duration. It does not expose a detached semantic K/V contract. Thus the stock daemon qualifies at E0. A future Ollama integration can reach E2 by delegating to a backend adapter with model/resource identity and native attention support; it should not duplicate backend kernels in the product layer.

4 Implementation

`OllamaEngineAdapter` calls `/api/show` before execution and hashes the model name, modification metadata, details, and model information. A model change invalidates state held by an optional native backend executor. In E0, selected resources are inserted as an ordinary system message with explicit resource IDs and URIs, then sent to `/api/chat`. Tool schemas and the request's generation bound are preserved. Reasoning is an explicit adapter option and defaults off for bounded answer generation. Ollama 0.30.8 honors `think=false` on its native endpoint; its OpenAI-compatible endpoint ignored that extension in our run, so the benchmark also sends Qwen's `/no_think` control. A regression test protects both the request field and prompt control and verifies that the caller's messages are not mutated. Ollama 0.32.7 with the official Qwen3-14B package still emitted reasoning-only OpenAI deltas. We therefore add a native NDJSON benchmark path that reads only user-visible `message.content`, captures prompt/evaluation counts, and sets `think=false`. The empty-content OpenAI pass is rejected rather than scored as zero quality.

The backend contract is `pra-engine/1`. A valid E2 receipt names llama.cpp and its revision, matches the model fingerprint returned by `/api/show`, binds the configured sidecar to a model-artifact digest, and asserts native K/V, unified-cache sequence attachment, metadata-only attachment, request-sequence cleanup, resource identity, tenant isolation, and cleanup. Missing mechanisms, a stale fingerprint, negotiation errors, or disappearance of the executor select E0. The adapter caches a valid receipt only for that fingerprint; model change and unload invalidate it.

`inspect_endpoint()` queries `/api/version`, `/api/tags`, and `/api/ps`. The runtime provider's doctor reports the endpoint and models. It labels native PRA as fallback unless a sidecar URL, pinned backend revision, and model-artifact digest are supplied and the sidecar proves `pra.llama.cpp/v1`. `unload()` uses Ollama's zero keep-alive request and invalidates delegated state. Unit tests cover E0 injection, endpoint inspection, metrics preservation, and the rule that only explicit backend delegation upgrades to E2.

5 Experimental Method

We ran Ollama 0.6.8 on Windows 10 with Qwen2.5-0.5B. The natural cohort freezes the evidence selection and compares no context, selected text up to 384 source tokens, and full text up to 1024 source tokens. It contains five examples each from HotpotQA, QASPER, and 2WikiMultiHopQA, with two repeats and 16 generated tokens. Streaming HTTP measurements provide TTFT and inter-token latency.

The lifecycle experiment unloads Qwen, runs one cold and five keep-alive requests, switches to SmolLM2-135M, unloads it, and returns to Qwen. The selected record and query remain fixed. This isolates model lifecycle from retrieval. The cohort is a smoke calibration and does not support production tail-latency or model-quality claims.

A second host is an Apple M5 with 10 CPU and 10 GPU cores and 16 GB unified memory, running Ollama 0.30.8 and the official Qwen3-8B Q4_K_M GGUF. The larger-model cohort uses the same 15 identities and frozen selections, a 384-token selected cap, a 4096-token full cap, two repeats, and 12 generated tokens. Its separate lifecycle run unloads Qwen3-8B, executes five warm requests, switches to SmolLM2-135M, and returns to Qwen.

Table 1: Natural E0 qualification on Ollama. Prompt ratio is selected/full; TTFT is median ms; F1 is selected/full.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.410	348.6 / 602.8	0.404 / 0.314
QASPER	0.375	300.0 / 732.7	0.217 / 0.099
2Wiki	0.517	309.2 / 263.3	0.036 / 0.136

Table 2: Qwen3 Metal scale replication. Prompt ratio is selected/full; TTFT is p50 ms; F1 is selected/full.

Model	Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
8B	HotpotQA	0.327	1129 / 1328	0.381 / 0.181
8B	QASPER	0.100	929 / 2533	0.113 / 0.108
8B	2Wiki	0.422	984 / 844	0.195 / 0.200
14B	HotpotQA	0.329	1044 / 1731	0.120 / 0.270
14B	QASPER	0.100	847 / 6257	0.086 / 0.110
14B	2Wiki	0.424	981 / 658	0.163 / 0.163

The first SmolLM use includes Metal runner compilation and is therefore a deployment observation, not model-scale evidence.

A third run uses an M4 Pro with 20 GPU cores, 48 GiB unified memory, and Ollama 0.32.7. Qwen3-14B is fully GPU resident. The same 15 frozen identities, 384/4096 source-token caps, two repeats, and 12-token generation bound are sent through native `/api/chat`; retrieval does not change from the 8B run.

The live delegation cohort runs on the same M4 Pro with Ollama 0.32.7 and a patched llama-server at commit `458681e1d5d`. Ollama's manifest and the sidecar both resolve to GGUF SHA-256 `c5396e06af29...`. Five controlled facts are repeated five times. Selection is frozen: E0 serializes the selected fact through `/api/chat`; E2 sends only the completion stem and attaches the same fact's native K/V by sequence membership. We report one post-unload Ollama reload separately, then steady E0, E2 with resource encoding, and E2 with the resource already resident. Semantic answer aliases accept equivalent numeric formatting; warm determinism compares generated token IDs exactly.

6 Results

Selected text substantially reduces visible context. It also improves TTFT and token F1 on HotpotQA and QASPER in this run. The 2Wiki reversal is equally important: the frozen selector's evidence and a small-model generation cap are not uniformly sufficient. E0 input compression is measured; universal quality preservation is not.

The 8B replication preserves the same qualitative boundary at a more realistic local-model scale. Selected text removes 57.8–90.0% of visible prompt tokens. It lowers median TTFT by 14.9% on HotpotQA and 63.3% on QASPER; 2Wiki is 16.6% slower at the median, while its full-context p95 reaches 2534 ms versus 1528 ms selected. F1 is approximately preserved on QASPER and 2Wiki and higher on the five Hotpot examples. Repeats contribute latency samples, not independent quality samples, so these signs remain smoke evidence.

The 14B point strengthens the transport result but not a selected-context quality claim. It removes 57.6–90.0% of visible prompt tokens and lowers median TTFT by 39.7% on HotpotQA and 86.5% on QASPER. The 2Wiki median is 49.1% slower selected, while its full-context p95 is 5.03 s versus 2.37 s selected. F1 is 0.120/0.270 and 0.086/0.110 selected/full on HotpotQA and QASPER, and equal at 0.163 on 2Wiki. These five-example slices show that input pressure and answer quality remain separate acceptance gates.

For Qwen3-8B on M5, cold-after-unload latency was 5836 ms and the five warm requests averaged 713.5 ms, an 8.18× reduction. All six returned the selected access code. The first SmolLM2 request took 40.20 s, dominated by 39.38 s reported load time; returning to Qwen took 753 ms. This one-time alternate-runner result should not be folded into steady-state PRA latency.

The warm mean is 4.54× faster than the initial cold request. Most of the difference is model load and prompt evaluation, not PRA logic. The switch test also shows why the adapter binds native state to a model fingerprint. Returning to Qwen was warm in this daemon configuration, but an adapter cannot assume that a runner survives every memory-pressure or scheduling event.

Table 3: Lifecycle measurements. Warm is the mean of five keep-alive requests.

Event	Total ms	Load ms	Prompt-eval ms
Qwen cold after unload	2300.5	1500.6	329.9
Qwen warm mean	506.5	30.4	5.9
Switch to SmolLM2	5248.7	4465.3	248.3
Return to Qwen	503.2	36.1	5.7

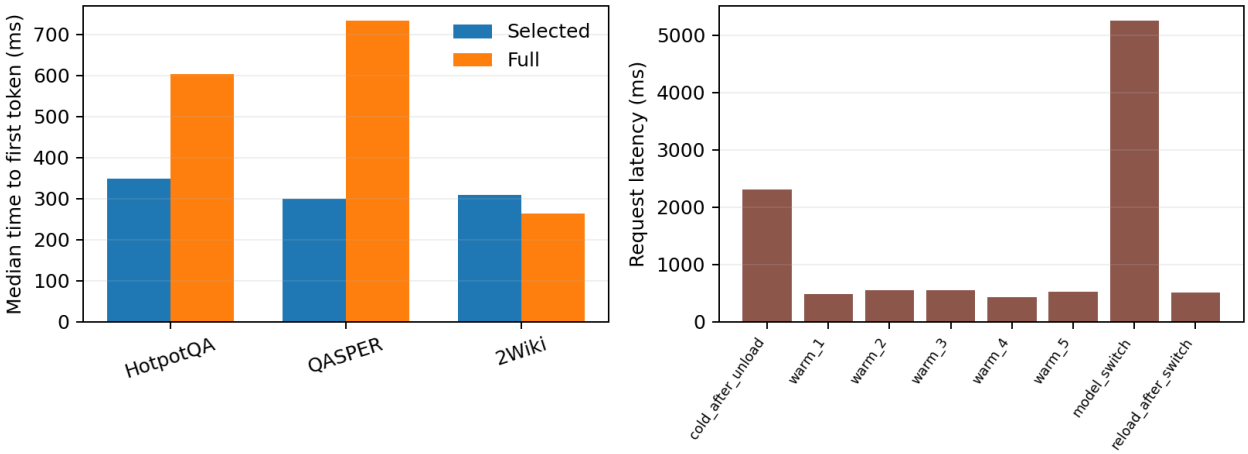

Figure 2: Selected/full context pressure (left) and model lifecycle (right). Ollama's keep-alive benefit is large enough that lifecycle state must be reported separately from PRA selection.

6.1 Delegated llama.cpp handshake

The inherited engine mechanism uses pinned llama.cpp commit `458681e1d5d`. A selected resource is encoded under a stable sequence in unified K/V storage and attached to a request by `llama_memory_seq_cp`; upstream changes cell membership rather than copying physical K/V. Five prompts on CPU and Metal produce 10/10 exact schedule-matched E0/E2 logits and four-step decode. Warm reattachment is exact in 10/10. An absent second request is exact on CPU and preserves the top token with maximum error below 0.006 on Metal.

We feed that mechanism identity into the Ollama control plane, then exercise five protocol cases. The complete receipt upgrades AUTO to E2 exactly once. A receipt containing only `native_kv` and a complete receipt bound to a stale model fingerprint both fall back to selected-text E0 and never invoke the native executor. Model switch invalidates the old fingerprint before a new receipt is accepted; unload invalidates the new receipt and returns capability reporting to E0.

Table 4: Backend negotiation and lifecycle controls. These are protocol controls over a model-backed llama.cpp mechanism receipt, not stock-Ollama E2 requests.

Case	AUTO result	Native invocation
Complete pinned receipt	E2	1
Missing sequence-attach mechanisms	E0	0
Stale model fingerprint	E0	0
Model switch	E2 after re-handshake	old state invalidated
Unload	E0	new state invalidated

6.2 Live product delegation

The negotiated executor now invokes the patched server rather than a mock receipt. All 25 requests negotiate E2. E0, first-use E2, and warm E2 each answer 25/25 facts correctly, and every warm E2 sequence exactly matches its first-use E2 sequence. The post-unload Ollama reload takes 722.8 ms and is excluded from steady E0. Mean

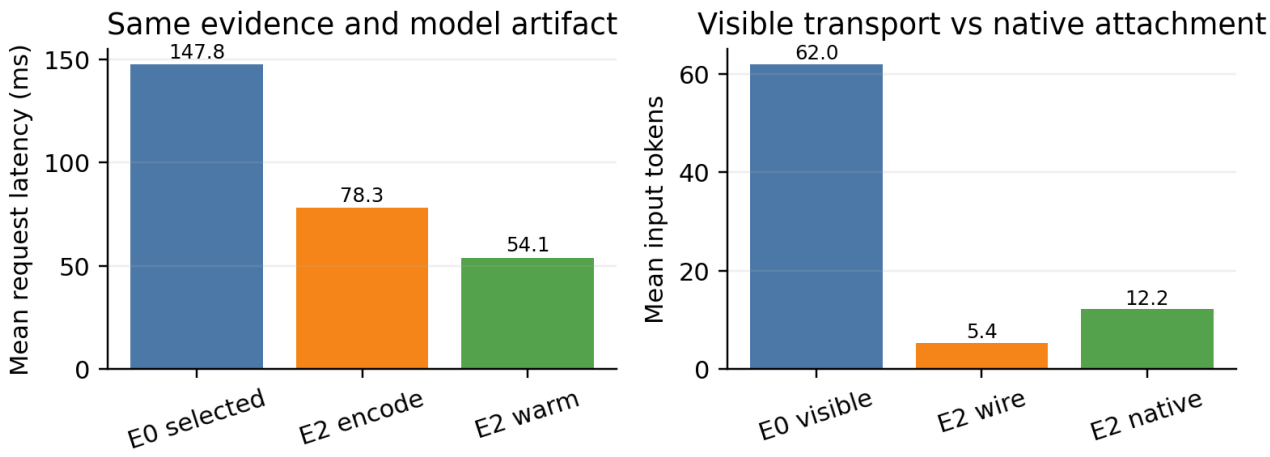

Figure 3: Live E0/E2 delegation on one exact Qwen2.5-0.5B GGUF. Left: steady latency, separating first resource encoding from warm reuse. Right: E0 visible prompt tokens versus E2 wire and attached-native tokens. All three conditions answer 25/25 controlled facts correctly.

request latency is 147.8, 78.3, and 54.1 ms respectively; first-use E2 is 0.530× E0 and warm E2 is 0.366× E0. The mean first-use minus warm difference is 24.3 ms, an empirical estimate of resource encoding in this cohort. P95 is 168.0, 83.8, and 58.4 ms.

E0 exposes 62.0 mean prompt tokens. E2 carries 5.4 mean wire tokens and attaches 12.2 native tokens without physical K/V copy, reducing visible transport by 91.3%. These counts describe different representations of the same selected fact; they should not be added and interpreted as an ordinary prompt length. The result demonstrates a local-AI path in which Ollama remains the catalog and fallback surface while a pinned backend consumes persistent native memory. It does not establish scheduler-managed E3, general natural-QA parity, or a speed advantage for larger contexts and models.

7 Deployment Matrix

Table 5: Current product status.

Mode	Transport	Status	Claim
E0 selected	Ollama/OpenAI HTTP	measured	supported default
E1 logical	optional envelope	interface only	no native effect
E2 native	delegated llama.cpp	live measured	pinned sidecar
E3 scheduler	Ollama runner	not implemented	no claim

This design keeps ordinary clients compatible. An application can point the PRA gateway at Ollama today. If a future active backend reports E2, the same wire request can carry stable identities while the backend performs native materialization. If the handshake disappears after an upgrade or model switch, AUTO returns to E0.

8 Limitations and Next Work

The original Windows runtime is old relative to the pinned source audit; the paper does not merge those observations with the separate 0.30.8 M5 and 0.32.7 M4 Pro replications. Three model sizes are measured, but all natural cohorts are small. Memory use, concurrent requests, cancellation, reconnects, daemon restart, and quantized variants remain open. The live E0/E2 cohort is controlled and small; it does not replace frozen-selection natural QA or establish that the stock Ollama runner itself carries selected resource identity. The sidecar currently uses one locked resource/request slot pair, so multi-tenant scheduling and resource-slot allocation remain engineering

work. The next experiment should hold natural selections fixed across both transports, then add concurrency and lifecycle failure injection without changing the client.

9 Conclusion

Ollama can provide a low-friction PRA entry point without becoming a second PRA runtime. The measured E0 path reduces visible context at 0.5B, 8B, and 14B; the larger Mac runs expose both substantial TTFT reductions and dataset-specific quality/reuse reversals. Keep-alive dominates cold latency. Native support must be delegated and negotiated. The pinned llama.cpp sidecar now provides a live E2 path with exact warm reuse and lower latency on a 25-request controlled cohort. On stock daemons the receipt remains absent and safe E0 fallback remains the default.

Artifact Availability

Branch `research/paper6-8-ollama` contains the adapter, pinned llama.cpp probe, tests, raw JSON, lifecycle benchmark, plot generator, README, and manuscript.

References

[1] Ollama contributors. *Ollama*. https://github.com/ollama/ollama

[2] OpenAI. *Chat Completions API*. https://platform.openai.com/docs/api-reference/chat

[3] Qwen Team. *Qwen2.5 Technical Report*. 2024. https://arxiv.org/abs/2412.15115